

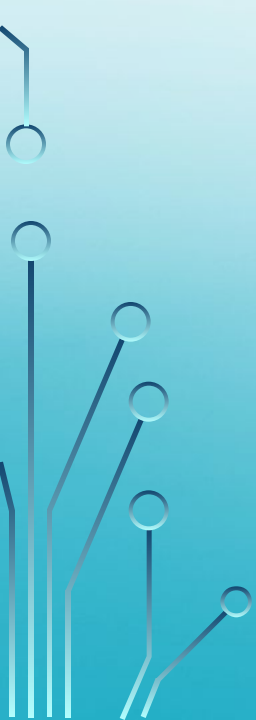
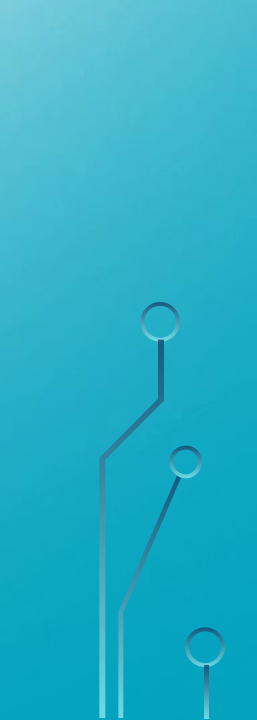
PYTHON DAY 2



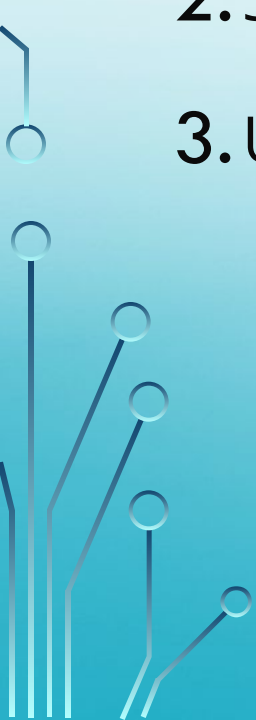
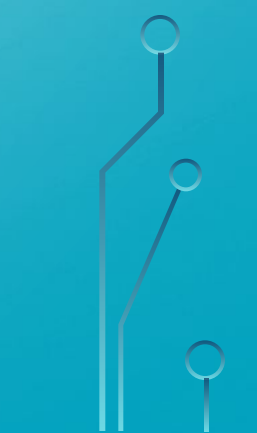
• Prepared by Isaac Kamau +254704205553 | [linkedin.com/in/isaak-mwangi/](https://www.linkedin.com/in/isaak-mwangi/)



• In this session, we are shall be covering:

- Python Line structure
 - Python comments
 - Python indentation
 - Python coding style
- 
- 
- 

- 
- As you learn Python throughout this course, there are a few things you should keep in mind.

1. Python is case-sensitive.
 2. Spacing is important.
 3. Use error messages to help you learn
- 
- 

PYTHON INDENTATION

- Indentation refers to the spaces at the beginning of a code line.
- Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.
- Python uses indentation to indicate a block of code.

PYTHON COMMENTS

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.
- `#This is a comment`

MULTILINE COMMENTS

- Since Python will ignore string literals that are not assigned to a variable, you can add a multiline string (triple quotes) in your code, and place your comment inside it:

- `"""`

```
This is a comment  
written in  
more than just one line  
"""
```

PYTHON CODING STYLE

- To write beautiful python code we use PEP 8 Style guide formatting.
- PEP 8, sometimes spelled PEP8 or PEP-8, is a document that provides guidelines and best practices on how to write Python code.
- Read more about PEP 8 here:
- <https://peps.python.org/pep-0008/>

PYTHON'S HELLO WORLD

- Python is an interpreted programming language, this means that as a developer you write Python (.py) files in a text editor and then put those files into the python interpreter to be executed.
- Let's write our first Python file, called helloworld.py, which can be done in any text editor.
- `print("Hello, World!")`

PYTHON VARIABLES

- **Variables:**
- Variables are containers for storing data values.
- **Creating Variables:**
- Python has no command for declaring a variable.
- A variable is created the moment you first assign a value to it.

EXAMPLE OF A VARIABLE

- `x = 5`
`y = "John"`
`print(x)`
`print(y)`

- Many Values to Multiple Variables
- Python allows you to assign values to multiple variables in one line:

- Example

- `x, y, z = "Orange", "Banana", "Cherry"`
`print(x)`
`print(y)`
`print(z)`

OUTPUT VARIABLES

- The Python `print()` function is often used to output variables.
- Example
- `x = "Python is awesome"`
`print(x)`

PYTHON OPERATORS

- + Addition
- - Subtraction
- * multiplication
- / division
- % mod (the remainder after dividing)
- ** Exponentiation
- // Divides and rounds down to the nearest integer

Comparison Operators

Symbol	Use Case	Bool	Operation
$5 < 3$		False	Less Than
$5 > 3$		True	Greater Than
$3 \leq 3$		True	Less Than or Equal To
$3 \geq 5$		False	Greater Than or Equal To
$3 == 5$		False	Equal To
$3 \neq 5$		True	Not Equal To

STRINGS

- Strings in Python are shown as the variable type (str) You can define a string with either double quotes (" ") or single quote (' ')

STRING METHODS

- **Accessing Values in Strings**

- We use square brackets for slicing along with the index or indices to obtain a substring.
- `var1 = "MANU!"`
- `var2 = "Software Testing"`
- `print ("var1[0]:",var1[0])`
- `print ("var2[1:5]:",var2[1:5])`

ANOTHER EXAMPLE

- `x = "Hello World!"`
- `print(x[:6])`
- `print(x[0:6] + "Manu")`

PYTHON STRING REPLACE() METHOD

- `oldstring = 'I like python'`
- `newstring = oldstring.replace('like', 'love')`
- `print(newstring)`

CHANGING UPPER AND LOWER CASE STRINGS

- `string="python at manu"`
- `print(string.upper())`

USING “JOIN” FUNCTION FOR THE STRING

- The join function is a more flexible way for concatenating string. With join function, you can add any character into the string.
- For example, if you want to add a colon (:) after every character in the string “Python” you can use the following code.

- `print(":".join("Python"))`

REVERSING STRING

- By using the reverse function, you can reverse the string. For example, if we have string "12345" and then if you apply the code for the reverse function as shown below.
- `string="1 2345"`
- `print("".join(reversed(string)))`

SPLIT STRINGS

- Split strings is another function that can be applied in Python let see for string “guru99 career guru99”. First here we will split the string by using the command `word.split` and get the result.
- `word="guru99 career guru99"`
- `print(word.split(' '))`

- To understand this better we will see one more example of split, instead of space (' ') we will replace it with ('r') and it will split the string wherever 'r' is mentioned in the string

- `word="guru99 career guru99"`

- `print(word.split('r'))`